



Open Source Runtime Security Tools in Azure Kubernetes Service

Radu-Constantin Onuțu

Dissertation · July 2026

University of Bucharest

Faculty of Mathematics and Computer Science

Supervisor: Lect. dr. Silviu Laurențiu Vasile

Motivation & Background

Why runtime security for Kubernetes

- Kubernetes is the dominant orchestrator: 82% of container-using organizations run it in production (CNCF 2025), up from 66% in 2023.
- Build-time (image scanning) and deploy-time controls (admission control) are *preventive* and stop once a container runs.
- At *runtime*, real threats remain: container escape, crypto-mining, lateral movement (MITRE ATT&CK for Containers).
- eBPF tools observe the kernel (syscalls, exec, file, network) at low overhead: no app changes, no kernel modules.

The practical problem

Three open-source eBPF tools lead the space (Falco, Tetragon, Tracee). Commercial suites (Sysdig Secure, Aqua) are costly, so teams want to pick an open-source one, but the tools differ and comparative evidence is thin.

Research gap and research questions

Closest prior work (Syairozi & Arizal, 2025): same three tools, DigitalOcean, three OWASP scenarios. Reported 100% detection, 0% false positives, but did not cover enforcement, alert quality, a MITRE-structured scenario set, a hyperscale managed platform, or operational complexity.

Gaps this thesis targets

- No evaluation on a **hyperscale managed** platform (AKS / EKS / GKE).
- No scenario set structured around **MITRE ATT&CK for Containers**.
- Detection, enforcement, alert quality, and operational complexity never combined in one study.

RQ1 How do the tools differ *architecturally* (collection, detection, response)?

RQ2 How do they compare on *detection, overhead, operational complexity* on AKS?

RQ3 What practical *tool-selection guidance* follows?

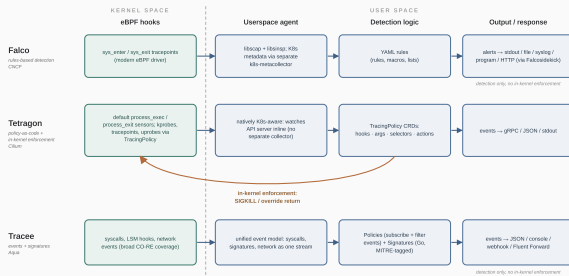
The three tools at a glance

(RQ1)

All three share one architecture: **eBPF programs on kernel hooks** feeding a **userspace agent**. They diverge on detection model and, decisively, on enforcement.

- **Falco**: rules-first, signature-based (YAML rules); detection-only. CNCF Graduated.
- **Tetragon**: policy-as-code (TracingPolicy CRDs); the **only** tool that enforces **in-kernel** (SIGKILL / override return). Cilium sub-project.
- **Tracee**: event collection + behavioural signatures (Go); built-in MITRE mapping; detection-only. Aqua Security.

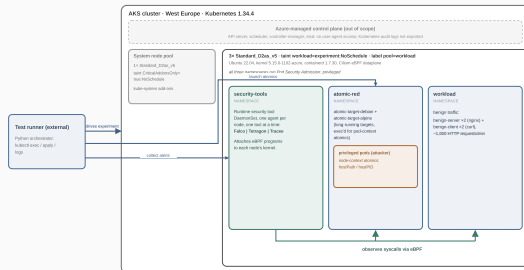
Key axis: only Tetragon can *block*, not just alert.



Methodology

Experimental setup on AKS

- Single **AKS** cluster (West Europe), Kubernetes 1.34.4, kernel 5.15.0-1102-azure, containerd 1.7.30.
- 1-node system pool + 3-node workload pool. Each tool runs as a **DaemonSet**, **one tool at a time** (overlapping kernel hooks would confound the comparison).
- Three namespaces: *security-tools* (the agent), *atomic-red* (targets + attackers), *workload* (benign HTTP traffic, ~1,000 req/min).
- **Identical resource envelope** for all three tools (requests 200m/512Mi, limits 2 CPU/2 GiB): fairness, not chart defaults.
- Audit logs out of scope, so the comparison is scoped to **node-level eBPF detection**.



Attack catalogue and metrics

Catalogue (MITRE ATT&CK Containers v19)

- One atomic per parent technique → 18 atomics across all 10 tactics.
- Drawn from Atomic Red Team (pinned commit); Docker-only atomics adapted to Kubernetes-native equivalents.
- Two contexts: *pod* (15, *kubectl exec* into a target) and *node* (3, privileged escape: T1611, T1613, T1685.004).
- 10 reps × 18 atomics = 180 runs/tool, 540 total; 30-min benign baseline before each run.

Seven metrics

1. Detection Rate (DR)
2. Mean Time to Detect (MTTD)
3. False-positive behaviour
4. Resource overhead (CPU / memory)
5. Alert quality & enrichment
6. Operational complexity
7. Runtime enforcement

Quantitative where measurable, qualitative for alert quality and operational complexity.

Results

Tool	Detection rate	Mean MTTD	Max MTTD
Falco	177/180 (98.3%)	best 1.10 s	2.48 s
Tetragon	164/180 (91.1%)	1.22 s	3.51 s
Tracee	179/180 (best 99.4%)	1.28 s	14.97 s

- Detection rates **cluster tightly**; mean MTTD differs by < 200 ms across all three (all around one second).
- **Tracee** has the highest rate but a **heavy tail** (p99 9.70 s, max 14.97 s) on aggregation-driven atomics.
- Lower than the reference paper's 100%, consistent with the broader catalogue (18 atomics / 10 tactics vs. 3 scenarios) and signals outside the tools' upstream packs.

Baseline alerts (30-min benign window)

Tool	Total	Residual
Falco	31,625	110
Tetragon	1,533	best 24
Tracee	6,458	2,843

Totals are dominated by **workload-generator artefacts**; the residual strips them out. **Zero** alerts in the attack namespace for all three.

Per-pod overhead (baseline mean)

Tool	CPU (m)	Mem (MiB)
Falco	22.8	best 93.1
Tetragon	best 18.5	201.6
Tracee	44.9	125.5

All three sit **within one order of magnitude**. Tetragon: lowest CPU, highest memory (eager kprobe / map allocation). Tracee: highest CPU (always-on collection).

The false-positive count is alert *volume* over a fixed benign workload, not a textbook FPR (the true-negative count is not observable in continuous monitoring).

Only **Tetragon** can block in-kernel (*matchActions* on a TracingPolicy selector). **Falco** and **Tracee** are detection-only.

Demonstration: T1611.1 container escape (*nsenter*)

Policy scoped to */usr/bin/nsenter*, SIGKILL on *sys_setns*.

- **10/10 blocked**: exit code 137, no post-*setns* output, matching policy event 10/10.
- End-to-end latency 2.13 / **2.58** / 3.03 s (min/mean/max), **dominated by pod scheduling**, not the in-kernel kill (microseconds).

Why it matters: enforcement removes the detect-then-respond race; two of three tools cannot do it. This asymmetry is itself a substantive cross-tool finding.

Cross-tool summary

Tool	DR	MTTD (s)	FP total	FP resid.	CPU (m)	Mem (MiB)	Alert q. (/14)	Enforce
Falco	177/180	1.10	31,625	110	22.8	93.1	14	–
Tetragon	164/180	1.22	1,533	24	18.5	201.6	10	10/10
Tracee	179/180	1.28	6,458	2,843	44.9	125.5	12	–

- No single tool wins every column: the choice is a **trade-off**.
- **Falco**: richest alerts, lowest latency, zero friction.
- **Tetragon**: only enforcement, lowest CPU and residual FP.
- **Tracee**: highest detection rate, lowest authoring cost.

Discussion & Conclusion

Choose Falco when

detection (not blocking) is the goal, alert metadata matters (severity + MITRE on every alert), and out-of-the-box coverage is preferred (14/18 atomics at install, zero deployment friction).

Choose Tetragon when

preventive **in-kernel blocking** is required and the team can invest in TracingPolicy authoring. Lowest CPU and residual FP; costs are highest authoring effort and no severity / MITRE tags.

Choose Tracee when

broad coverage at **low authoring cost** is the priority and the highest detection rate is sought. Costs are the highest baseline FP residual and CPU footprint.

Conclusion and future work

Findings. On AKS, across 540 attack repetitions and 18 atomics spanning all 10 MITRE ATT&CK Containers tactics, the three tools detect at **comparable rates** (91–99%) with **~1 s** response, but differ materially on policy authoring effort, alert metadata, false-positive profile, and **in-kernel enforcement**.

Contributions

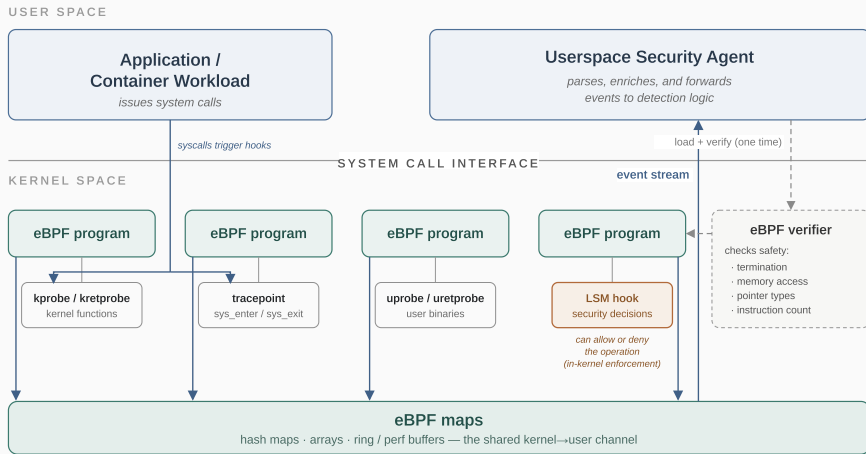
1. First comparison of these three tools on a **hyperscale managed** platform (AKS).
2. **Unifies** detection, enforcement, alert quality, and operational complexity in one evaluation.
3. Scenario set structured by **MITRE ATT&CK for Containers** (10/10 tactics).

Future work. Integrate the audit-log stream; compose tools (Tetragon enforce + Falco/Tracee detect); extend ***matchActions*** enforcement to composite-signal atomics.

Thank you.

Questions?

Backup: eBPF two-part architecture



CO-RE + BTF: one compiled program runs unchanged across kernel versions.

—> runtime event flow
--> load + verify (one time)

Backup: per-atomic detection rate

Tactic	Atomic	F	Te	Tr
Initial Access	T1078.003	10	10	10
Execution	T1059.004	9	9	10
Execution	T1053.002	10	8	10
Persistence	T1098.004	10	10	9
Persistence	T1136.001	10	10	10
Persistence	T1543.002	10	9	10
Priv. Esc.	T1611	9	10	10
Stealth	T1070.003	10	10	10
Stealth	T1036.004	10	9	10

Tactic	Atomic	F	Te	Tr
Def. Impair.	T1685.004	10	9	10
Cred. Access	T1110.001	10	7	10
Cred. Access	T1552.007	10	10	10
Discovery	T1046	10	8	10
Discovery	T1069.001	9	10	10
Discovery	T1613	10	8	10
Lateral Mvmt.	T1550.001	10	8	10
Impact	T1485	10	9	10
Impact	T1496	10	10	10
Total		177	164	179

Detected reps out of 10. F = Falco, Te = Tetragon, Tr = Tracee.

Backup: MTTD distribution per tool

Tool	<i>n</i>	Mean	Median	Std	p95	p99	Max
Falco	177	1.10	0.95	0.39	2.01	2.44	2.48
Tetragon	164	1.22	1.07	0.54	2.25	2.69	3.51
Tracee	179	1.28	0.87	1.96	4.04	9.70	14.97

All values in seconds, over detected repetitions. Percentiles use the nearest-rank method. Falco and Tetragon cluster around one second; Tracee is bimodal (sub-100 ms on direct kernel-event hooks, multi-second on aggregation-driven atomics), which inflates its tail and standard deviation. Node-context atomics (T1611, T1613, T1685.004) rise above 1.5 s for all three because the clock starts before the privileged pod is scheduled, not because of slower agent processing.

Backup: alert quality (7 fields)

Field	Falco	Tetragon	Tracee
Rule / signature identifier	Present	Present	Present
Severity	Present	Absent	Partial
Process name + command line	Present	Present	Present
File path / network endpoint	Present	Present	Present
Pod name + namespace	Present	Present	Present
Container identifier	Present	Present	Present
MITRE ATT&CK technique tag	Present	Absent	Partial
Composite (out of 14)	14	10	12

The two differentiating fields are **severity** and the **MITRE tag**. Falco emits both on every alert. Tetragon emits neither (left to downstream SIEM). Tracee emits both only on signature findings, not on the plain Policy events used here, hence the partial scores.

Backup: operational complexity and deployment

Dimension	Falco	Tetragon	Tracee
Deployment friction	None	Raise memory 1→2 GiB; rename colliding policy names	Apply CRDs separately; readiness probe flaps
Policy authoring	7 custom rules (Falco rule language)	9 TracingPolicies (kprobe attachments)	9 Policy CRDs (subscribe to built-in events)
Coverage at install	14/18 atomics	0/18 (telemetry only)	6/18 atomics

Authoring counts are similar in scale but differ in nature: a Tetragon policy is closer to writing a small eBPF program (pick a kernel function, declare argument types, add matchers), whereas a Tracee Policy CRD only filters an existing event. Tetragon ships **no detections** by default: a first-time user gets a firehose of telemetry until a policy is authored.

Backup: scope and threats to validity

Deliberately out of scope

- **Audit-log-only** techniques (e.g. *kubectl exec*, Deploy Container): AKS Diagnostic Settings not enabled (cost). Node-side variants included instead (e.g. T1613 via *crictl*).
- Network-perimeter, registry/admission, real-CVE, human-in-the-loop, and image-build techniques: detection surface lies outside the eBPF kernel-hook layer.

Threats to validity

- *Internal*: default configs differ; equalised by giving each tool the config a first-time user reaches plus the rules/policies needed for catalogue coverage. Tetragon's lower DR reflects the authored policy set, not its ceiling.
- *External*: one AKS cluster, one kernel; generalisation to EKS/GKE/CRI-O is hypothesis, not result.
- *Construct*: FP is alert volume, not textbook FPR; MTTD includes a small uniform measurement overhead.